

JB Modular Character Builder

User Guide

What it does

JB Modular Character Builder is a Unity Editor tool for assembling modular characters from source prefabs or scene objects.

It lets you:

-  load one or more source characters
-  switch modular parts by character section
-  assign materials per slot
-  use custom colors or existing material assets
-  randomize character parts and material colors
-  save and load presets
-  export a clean prefab with generated materials

Location

Tools > João Baltieri 3D > JB Modular Character Builder

Basic workflow

1. Load sources

In Source:

-  assign one or more source characters
-  click Load to create the preview
-  use Rebuild to rebuild while keeping the current setup
-  use Restore Base to restore original slot materials
-  use Clear to remove the preview

2. Choose character parts

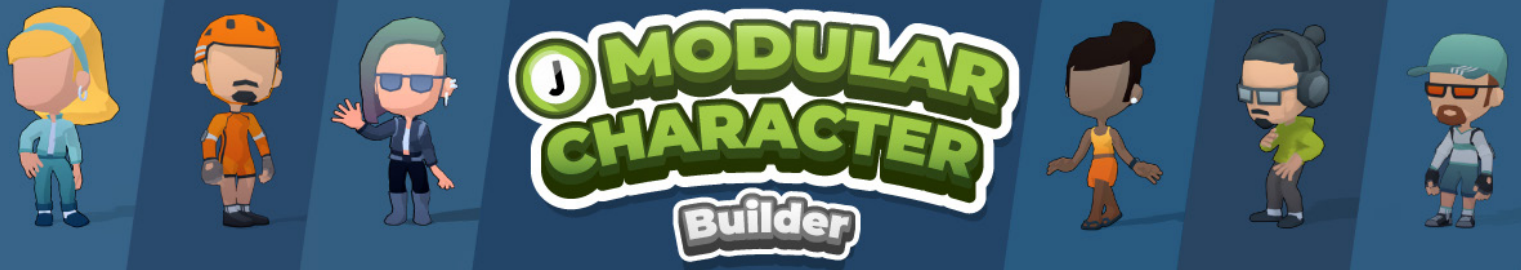
In Character Parts:

-  choose visible parts by category
-  some categories are single-select
-  some categories support multiple pieces
-  use locks to protect categories or pieces from randomization
-  use Randomize Character to generate variations

3. Edit materials

In Materials, each slot can work in one of two modes:

-  **Custom Color**
-  **Material Asset**



Custom Color

Available controls:

-  Base Color
-  Emission Color (HDR) for emissive slots

Notes:

- Selecting a material from another family changes the slot's active family
- Custom Color and Randomize Colors respect that active family
- Reset restores the original base family/material

Material Asset

Uses a real material asset directly.

Available controls:

-  drag a material into the field
-  click Browse... to open the material browser

Additional per-slot tools:

-  Copy
-  Paste
-  Reset
-  lock / unlock
-  Apply to All Body Parts (body slots only)

Material Browser

The material browser supports:

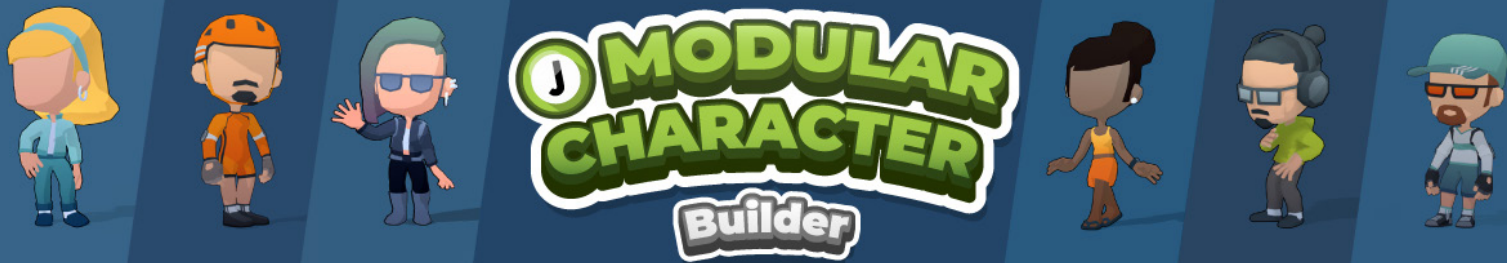
-  family tabs
-  favorites
-  Clear Favorites
-  recent materials
-  Clear Recent
-  search by name / shader / path
-  folder filtering
-  thumbnail grid preview
-  preview panel
-  Find in Project
-  Apply Material

Behavior:

- 1 click on a thumbnail applies the material instantly
- 2 clicks apply and close the popup

Recent items behave the same way

- Apply Material works as a final confirmation and closes the popup
- Favorites view shows all favorite materials grouped by family
- Materials in the grid are organized by color for faster browsing



Randomize Colors

Randomize Colors:

- affects unlocked slots
- respects slot locks
- respects the slot's current active family
- keeps body-related parts visually consistent
- supports family-aware color behavior
- **Lock Body / Unlock Body** lets you protect all body material slots at once before randomizing.

Sync / Free Body Colors

The Body Colors toggle controls how random color generation behaves for body-related material slots.

Sync Body Colors

All unlocked body slots share the same randomized base color.

This is useful for keeping the character body visually consistent, especially when the body is split across multiple meshes or material slots.

Free Body Colors

Each unlocked body slot is randomized independently.

This allows more variation between body parts and can produce more experimental results.

Important:

This option only affects Randomize Colors behavior.

It does not force materials to become the same manually, and it does not override locked slots.

Reset behavior

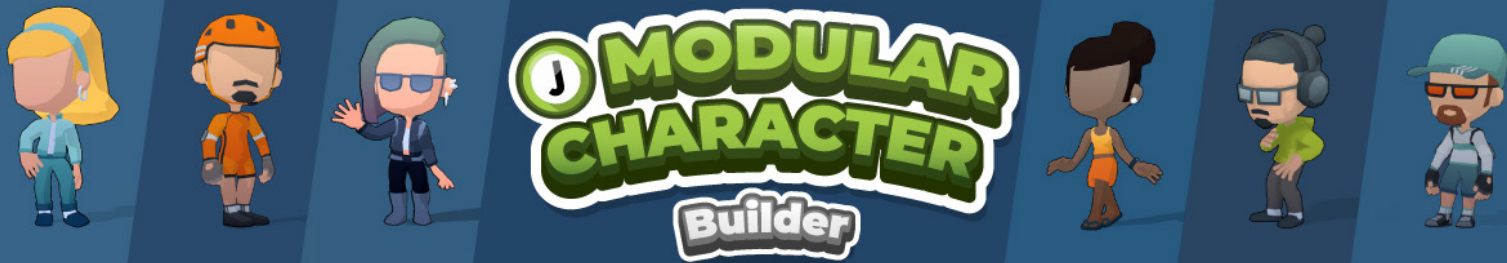
Slot Reset

- Each slot has its own Reset button.
- Reset All
- Reset All restores all slot material states in the preview, including slots belonging to inactive pieces.

Copy / Paste

Each slot can copy and paste its full state, including:

- custom vs material mode
- active material asset
- base color
- HDR emission color



Presets

Presets store:

- source character names
- category selections
- locks
- per-slot material state
- active family behavior
- custom colors
- HDR emission colors
- selected material assets

⚠ The UI also shows when the current state has changed since the last preset load/save.

Export

Export creates:

- a clean prefab
- generated materials only when needed
- deduplicated generated materials
- no dependency on runtime material scripts
- inactive objects removed from the final prefab

After export, the tool shows a success summary with:

- prefab path
- materials folder
- generated derived materials count
- reused derived materials count

Default folders

The tool creates and uses these folders inside the detected tool root:

```
<tool root>/
├── JB Materials/
├── JB Prefabs/
└── JB Presets/
```

Examples:

Assets/JB Modular Character Builder/JB Materials

Assets/JB Modular Character Builder/JB Prefabs

Assets/JB Modular Character Builder/JB Presets

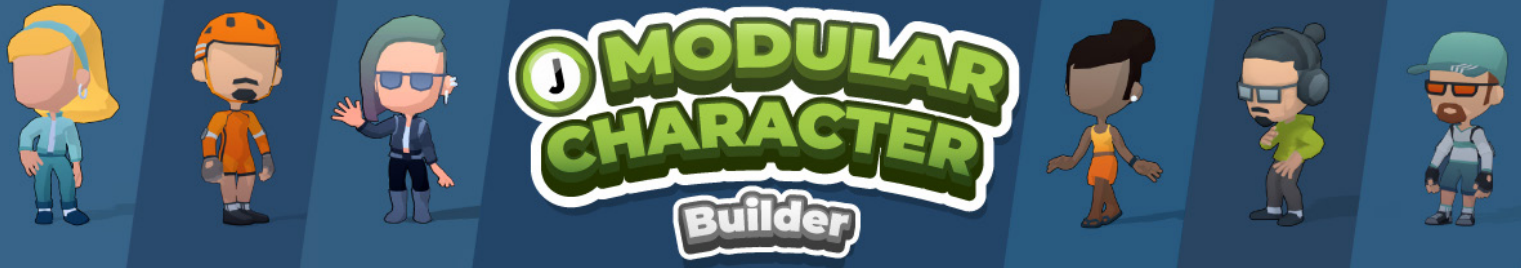
⚠ Important

The tool detects its own installation root dynamically, so it can still work even if the package is placed in another folder under `Assets`. You can also choose/create your preferred folder structure.

 **generated materials** go into **JB Materials**

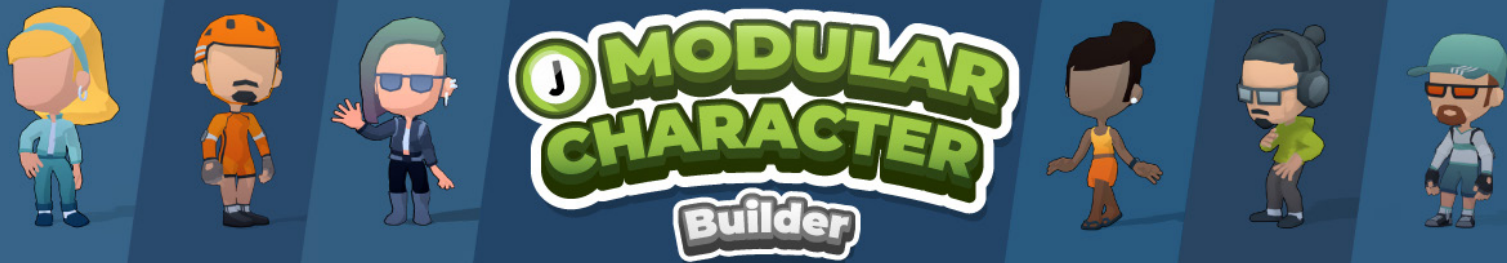
 **exported prefabs** go into **JB Prefabs**

 **presets** go into **JB Presets**



✓ **Current strengths**

- ◆ modular character assembly
- ◆ per-slot material workflow
- ◆ custom color + material asset modes
- ◆ active family-aware custom materials
- ◆ favorites / recents browser flow
- ◆ HDR emissive workflow
- ◆ preset system
- ◆ clean prefab export
- ◆ deduplicated generated materials



Technical information

Render pipeline compatibility

The tool is designed to work with common Unity material workflows used in: URP / HDRP

Built-in / Standard-style setups used as fallback-compatible materials

The material system checks common shader properties such as:

- base color
- emission color
- transparency / surface type
- metallic-related values

This allows the tool to behave correctly across most stylized character setups using standard Unity-compatible shaders.

Core system overview

The builder works around three main systems:

- Character Parts → modular piece visibility and selection
- Materials → per-slot material asset or custom color workflow
- Presets → saved build states for reuse

The tool is designed to work with modular characters that follow a consistent naming convention.

Default material families

The current default families are:

- Opaque
- Transparent
- Metallic
- Emissive

These families are used by:

- the material browser
- custom color mode
- randomize colors
- derived material generation
- export / deduplication

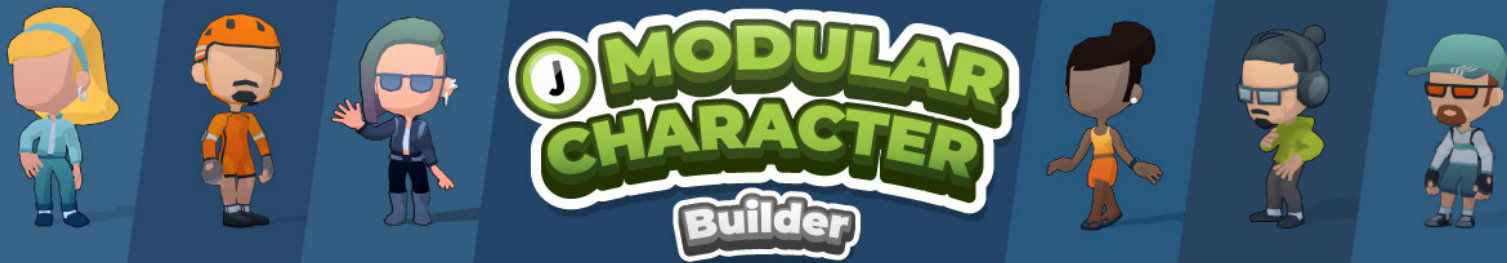
Important behavior

A slot has:

- a base/original family detected from the loaded source material
- a current active family based on the material currently chosen by the user

That means:

- if a slot originally loads as Opaque but the user assigns a Metallic material the slot will now behave as Metallic for custom color, randomization, preset save/load, and export
- Reset restores the slot back to its original base family/material.



Creating new materials for the tool

You can create and use new materials with the builder.

For best compatibility, materials should expose the standard properties the tool expects to read:

Recommended exposed properties:

Base Color:

- ``_BaseColor`` or ``_Color``

Emission Color

- ``_EmissionColor``

Surface / Transparency

- ``_Surface`` or ``_SurfaceType``

Metallic

- ``_Metallic``, ``_Metalness``, ``_MetallicScale``, or similar standard metallic controls

Optional texture properties used by custom-color workflow

If a material includes a base texture, the tool can disable that texture when switching to Custom Color.

For that behavior, the material should use a common base texture property such as:

- ``_BaseMap``
- ``_BaseColorMap``
- ``_MainTex``

Best practice

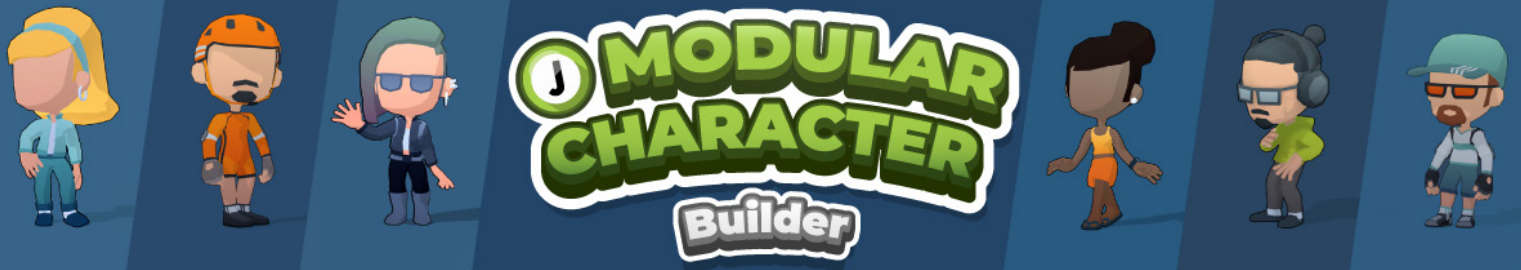
For the most predictable results, create materials using standard Unity-compatible Lit setups in URP or HDRP, or equivalent shaders that expose the properties above.

Default character part categories

The builder includes these default categories:

- Body
- HeadTop
- Hair
- Face
- Torso
- TorsoOuter
- TorsoGear
- Arms
- ArmsOuter
- ArmsGear
- Legs
- LegsOuter
- LegsGear
- Feet

These are the built-in categories the UI expects and sorts first.



Built-in multi-select categories

By default, these categories support multiple active pieces at the same time:

- Face
- TorsoGear
- ArmsGear
- LegsGear

Hidden empty categories

After loading the character, categories that do not contain any detected pieces are not shown in the UI. This keeps the builder cleaner and prevents empty sections from appearing when a source character does not use all default categories.

+ Extra categories

The tool also supports extra categories automatically.

If your character contains pieces with a category name not listed in the default set, the builder will still detect and display them.

Examples:

- `BackGear\_Backpack`
- `Accessory\_Pouch`
- `ShoulderGear\_Spikes`
- `HeadGear\_Helmet`

These extra categories are:

- scanned automatically from the source hierarchy
- added to the UI
- preserved in presets
- supported by export

Extra multi-select categories

If you want an extra category to behave as multi-select, type its name in the Extra Multi Categories field before loading or rebuilding.

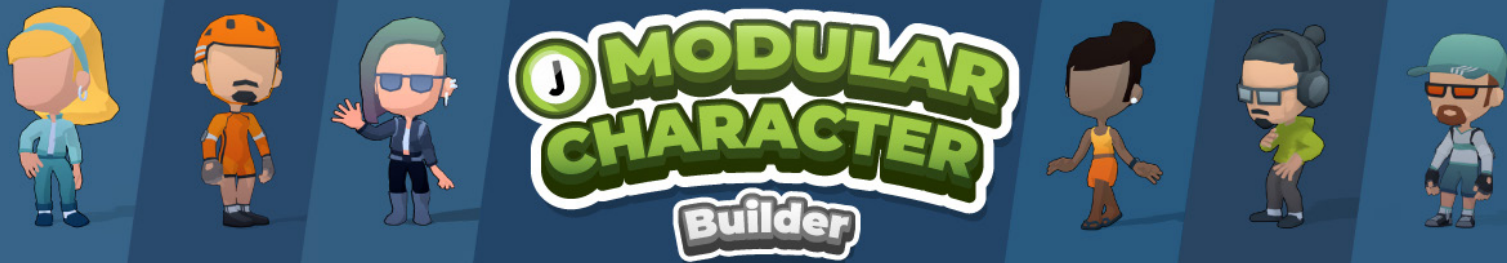
Example:

- BackGear, Accessory, ShoulderGear

This means those categories will allow multiple active pieces, just like the built-in multi-select categories.

Notes

- use a comma-separated list
- category names must match the parsed category names from your object naming
- this setting is useful before Load or Rebuild



Naming convention

The builder relies on naming convention to detect categories and variants.

Standard pieces

Format:

- Category_VariantName

Examples:

- Hair_Short
- Torso_Jacket
- Feet_Boots
- Face_Glasses

Body pieces

Body-related pieces use a special format:

Body_Variant

Body_SubCategory_Variant

Examples:

- Body_Head
- Body_Torso
- Body_Pelvis, etc..

How parsing works

- the first token defines the category
- the remaining tokens define the variant
- body pieces are treated specially so `Body\_\*` categories stay grouped correctly

This is why consistent naming is important for the builder to detect:

- category
- variant display name
- body grouping
- conflict handling
- preset restoration

Main files

- `ModularCharacterPrefabBuilderWindow.cs` (Editor)
- `MaterialBrowserPopupWindow.cs`
- `ModularCharacterBuildPreset.cs`

More infos, assets and exclusive content:

 joaobaltieri.com 

Contact/Support

joaobaltieri@gmail.com

Get the latest version here:

 github.com/joaobaltieri/JB-Modular-Character-Builder 